LAB – 16 | THEORY OF AUTOMATA & FORMAL LANGUAGES (BSC-H)

C File Handling
Reading & Writing Functions

A detailed reference document analysing all seven lab programs and every standard C library function used for file I/O operations.

SUBJECT	LAB	UNIVERSITY	SEMESTER	DATE
TA (BSC-H)	LAB-16	Darshan University	4	March 2026

Table of Contents

1. Introduction to File Handling in C

2. File Operation Lifecycle

3. fopen() – Opening a File

4. fclose() – Closing a File

5. Writing Functions

5.1 fprintf()

5.2 fputc()

5.3 fputs()

6. Reading Functions

6.1 fgetc()

6.2 fgets()

6.3 fscanf()

7. File Opening Modes Reference Table

8. Program-by-Program Analysis

a1.c – Write String to File

a2.c – Read File Character by Character

a3.c – Append Data to File

a4.c – Search Word in File

b1.c – Merge Two Files

b2.c – Count Words in File

b3.c – Store Student Records

9. Function Comparison at a Glance

10. Common Errors & Best Practices

11. Summary

1. Introduction to File Handling in C

File handling in C allows a program to **persist data** beyond program execution by reading from and writing to files stored on disk. The C standard library (`<stdio.h>`) provides a rich set of functions that abstract low-level OS system calls into portable, easy-to-use routines.

All file I/O in C revolves around the **FILE** structure — an opaque type defined in `<stdio.h>` that contains all the metadata the C runtime needs to manage an open file (buffer, position indicator, error flags, etc.). Programs never access this structure directly; they always use a *pointer* to it: `FILE *fp;`.

> **Key header required**
>
> Every file-handling program must include `#include <stdio.h>`. This header declares the FILE type, EOF macro, and all I/O functions discussed in this document.

2. File Operation Lifecycle

Every file interaction in C follows a strict three-step lifecycle:

① Open the file — `fopen()`

② Read / Write data — `fgetc, fgets, fscanf, fprintf, fputc …`

③ Close the file — `fclose()`

> ⚠ **Always check for NULL after fopen()**
>
> If `fopen()` fails (file missing, permission denied, disk full) it returns `NULL`. Attempting to read/write through a NULL pointer causes *undefined behaviour* — always validate first.

3. fopen() – Opening a File

OPEN

Syntax

```
FILE *fopen(const char *filename, const char *mode);
```

Parameters

Parameter	Type	Description
filename	const char *	Path/name of the file to open (e.g. `"output.txt"`)
mode	const char *	String specifying how the file should be opened (see §7)

Return Value

Returns a `FILE *` pointer on success, or `NULL` on failure.

Example (from a1.c)

```
FILE *fp;
fp = fopen("output.txt", "w");   // open for writing
if (fp == NULL) {
    printf("File cannot be opened!\n");
    return 1;
}
```

4. fclose() – Closing a File

CLOSE

Syntax

```
int fclose(FILE *fp);
```

Description

Flushes any pending writes in the buffer to disk, releases all resources associated with the stream and invalidates the pointer. Returns `0` on success, `EOF` on error.

> ✅ **Best practice**
>
> Always call `fclose()` for every successfully opened file. Forgetting to close a file may result in data loss (unflushed buffers) and resource leaks (each process has a limited number of open file descriptors).

```
fclose(fp);   // seen in every program: a1.c, a2.c, a3.c, a4.c, b1.c, b2.c, b3.c
```

5. Writing Functions

5.1 fprintf() WRITE

`fprintf()` is the file-writing counterpart of `printf()`. It writes a **formatted string** to a file stream using the same format specifiers as `printf`.

Syntax

```
int fprintf(FILE *fp, const char *format, ... );
```

Parameter	Description
fp	Pointer to the open FILE (must be opened in write/append mode)
format	Format string with optional conversion specifiers (%s, %d, %f, ...)
...	Variable arguments matching the format specifiers

Return: Number of characters written, or a negative value on error.

Usage in Lab Programs

```
// a1.c – write a string
fprintf(fp, "%s", str);

// a3.c – append a line
fprintf(fp, "%s\n", str);

// b3.c – write structured record
fprintf(fp, "%s %d %.2f\n", s.name, s.id, s.grade);
```

> 💡 **fprintf vs printf**
> The only difference between `fprintf` and `printf` is the first argument: `printf` always writes to *stdout*, while `fprintf` accepts any FILE pointer — including `stdout` and `stderr`.

5.2 fputc() WRITE

Writes a **single character** to the specified file stream. It is the simplest write function and is ideal when copying files character by character (as in b1.c).

```
int fputc(int ch, FILE *fp);
```

Parameter	Description
ch	Character to write (passed as int, internally cast to unsigned char)
fp	Pointer to the target file stream

Return: The written character on success, EOF on failure.

```
// b1.c — copy file1 into merged.txt one character at a time
while ((ch = fgetc(f1)) ≠ EOF) {
    fputc(ch, f3);   // write each character to output file
}
```

5.3 fputs() WRITE

Writes a **null-terminated string** (without the trailing `\0`) to the file. It does *not* automatically append a newline.

```
int fputs(const char *str, FILE *fp);
```

Return: Non-negative value on success, EOF on error.

> **Note**
> None of the seven lab programs use `fputs()` directly, but it is the character-string counterpart of `fgets()` and is commonly used with `fgets()`-based reading loops.

6. Reading Functions

6.1 fgetc() READ

Reads and returns the **next single character** from the file stream, advancing the file position indicator by one byte. Returns EOF (typically –1) when the end of file is reached or an error occurs.

```
int fgetc(FILE *fp);
```

Return value	Meaning
Character (0–255)	Successfully read character
EOF (–1)	End of file reached, or I/O error

Usage in Lab Programs

```
// a2.c – print every character of input.txt
while ((ch = fgetc(fp)) ≠ EOF) {
    printf("%c", ch);
}

// b1.c – copy characters from file1 & file2 into merged.txt
while ((ch = fgetc(f1)) ≠ EOF) {
    fputc(ch, f3);
}
```

> ⚠ **Store result in int, not char**
> `fgetc()` returns an `int` so it can represent both all possible byte values (0–255) *and* the special EOF (–1) sentinel. Storing the return value in a `char` variable can hide the EOF signal if `char` is unsigned on the target platform.

6.2 fgets() READ

Reads at most n–1 characters from the file into a buffer, stopping early if a **newline** or **EOF** is encountered. The newline, if read, is stored in the buffer. A null terminator is always appended.

```
char *fgets(char *buffer, int n, FILE *fp);
```

Parameter	Description

buffer	Destination character array
n	Maximum characters to read (including null terminator)
fp	Source file stream

Return: `buffer` on success, NULL at EOF or error.

Usage in Lab Programs

```
// a4.c – read file line by line to search for a word
while (fgets(line, sizeof(line), fp)) {
    lineNumber++;
    if (strstr(line, word) != NULL) {
        printf("Word found at line %d\n", lineNumber);
    }
}
```

> ✅ **fgets is safer than gets()**
>
> `fgets()` accepts a size argument and will never overflow the buffer — unlike the now-deprecated `gets()` function used in a1.c/a3.c for *stdin* input, which has no bounds check and is a classic security vulnerability (buffer overflow).

6.3 fscanf() READ

The file-reading counterpart of `scanf()`. Reads **formatted input** from a file, parsing tokens according to format specifiers, and skipping leading whitespace (spaces, tabs, newlines) by default.

```
int fscanf(FILE *fp, const char *format, ... );
```

Return: Number of items successfully matched and assigned. Returns EOF when end-of-file is reached before any match.

Usage in Lab Programs

```
// b2.c – count words in a file
while (fscanf(fp, "%99s", word) == 1) {
    count++;
}
// Return values:
//   1   → one word successfully read ✅
//   EOF → end of file reached        ❌
//   0   → input did not match format ❌
```

> 💡 **Why compare to 1 and not EOF?**
>
> Comparing `fscanf(...) == 1` is more robust than `!= EOF` because it also catches format-mismatch failures (return value 0), whereas checking only for EOF would loop forever on a malformed file.

7. File Opening Modes Reference Table

Mode	Meaning	File Exists	File Missing	Used In
`"r"`	Read only	Opens it	Returns NULL	a2.c, a4.c, b1.c, b2.c
`"w"`	Write only	Truncates to empty	Creates new	a1.c, b1.c, b3.c
`"a"`	Append only	Writes to end	Creates new	a3.c
`"r+"`	Read + Write	Opens it	Returns NULL	—
`"w+"`	Read + Write	Truncates	Creates new	—
`"a+"`	Read + Append	Reads from start, writes at end	Creates new	—
`"rb"`	Read binary	Opens it	Returns NULL	—
`"wb"`	Write binary	Truncates	Creates new	—

Text vs Binary mode

On Windows, text mode (`"r"`, `"w"`) translates `\r\n` ↔ `\n`. Binary mode (`"rb"`, `"wb"`) performs no translation, making it mandatory for non-text data (images, executables) and portable for struct-based I/O.

8. Program-by-Program Analysis

a1.c — Write a User String to a File

Functions: fopen · fprintf · fclose

Purpose: Accepts a string from the user and writes it to `output.txt`.

```c
// Full source — a1.c
#include <stdio.h>

int main() {
    FILE *fp;
    char str[100];

    printf("Enter a string: ");
    gets(str);          // reads from stdin (use fgets for safety)

    fp = fopen("output.txt", "w");

    if (fp == NULL) {
        printf("File cannot be opened!");
        return 1;
    }

    fprintf(fp, "%s", str);  // write string to file

    fclose(fp);
    printf("Data written to file successfully!");
    return 0;
}
```

Function	Role	Mode Used
fopen()	Open output.txt for writing (creates if not exists)	"w"
fprintf()	Write user-entered string with format specifier %s	—
fclose()	Flush buffer and close file	—

> ⚠ **Caution**
>
> `gets()` does not perform bounds checking and is removed in C11. Replace with `fgets(str, sizeof(str), stdin)`.

a2.c — Read a File Character by Character

Functions: fopen · fgetc · fgets · fscanf (shown) · fclose

Purpose: Reads and prints every character of `input.txt`. Also shows (commented out) how `fgets` and `fscanf` can do the same job differently.

```c
#include <stdio.h>

int main() {
    FILE *fp;
    char ch, str[100];

    fp = fopen("input.txt", "r");
    if (fp == NULL) { printf("File cannot be opened!"); return 1; }

    // Method 1 - character by character
    while ((ch = fgetc(fp)) != EOF) {
        printf("%c", ch);
    }

    // Method 2 (commented) - line by line
    // while (fgets(str, sizeof(str), fp) != NULL) {
    //     printf("%s", str);
    // }

    // Method 3 (commented) - word by word
    // while (fscanf(fp, "%s", str) != EOF) {
    //     printf("%s ", str);
    // }

    fclose(fp);
    return 0;
}
```

Method	Function	Granularity	Stops when
Active	fgetc()	One character per iteration	Returns EOF
Commented	fgets()	One line per iteration	Returns NULL
Commented	fscanf()	One whitespace-delimited word	Returns EOF

a3.c — Append Data to an Existing File

Functions: fopen("a") · fprintf · fclose

Purpose: Opens `log.txt` in **append mode** and adds a new line without overwriting existing content.

```
fp = fopen("log.txt", "a");    // "a" preserves existing content
fprintf(fp, "%s\n", str);     // \n ensures each entry is on its own line
fclose(fp);
```

> ✅ **Difference between "w" and "a"**
>
> Mode `"w"` destroys existing file contents. Mode `"a"` always positions the write cursor at the *end* of the file, making it perfect for log files that must grow over multiple program runs.

a4.c — Search for a Word in a File

Functions: fopen · fgets · strstr · fclose

Purpose: Reads `text.txt` line by line using `fgets()`; uses `strstr()` from `<string.h>` to search for a keyword and report its line number.

```
while (fgets(line, sizeof(line), fp)) {
    lineNumber++;
    if (strstr(line, word) != NULL) {
        printf("Word found at line %d\n", lineNumber);
        found = 1;
    }
}
// strstr(haystack, needle) - returns pointer to first match, or NULL
```

Key insight: `fgets()` preserves newlines, so each call maps exactly to one line of the file — ideal for line-number tracking.

b1.c — Merge Two Files into One

Functions: fopen(×3) · fgetc · fputc · fclose(×3)

Purpose: Opens `file1.txt` and `file2.txt` for reading, and `merged.txt` for writing. Copies each source file character-by-character into the destination.

```
FILE *f1, *f2, *f3;
f1 = fopen("file1.txt", "r");
f2 = fopen("file2.txt", "r");
f3 = fopen("merged.txt", "w");

while ((ch = fgetc(f1)) ≠ EOF) fputc(ch, f3);
while ((ch = fgetc(f2)) ≠ EOF) fputc(ch, f3);

fclose(f1); fclose(f2); fclose(f3);
```

This demonstrates that **multiple FILE pointers** can be open simultaneously. The fgetc→fputc pipeline is the simplest portable file-copy mechanism in C.

b2.c — Count Words in a File

Functions: fopen · fscanf · fclose

Purpose: Counts the total number of whitespace-delimited words in `file.txt` using `fscanf()`.

```
while (fscanf(fp, "%99s", word) == 1) {
    count++;
}
printf("Total words = %d", count);
```

fscanf Return	Meaning	Loop action
1	Word successfully read	Continue, increment count
EOF	End of file	Exit loop
0	Format mismatch	Exit loop (safe)

> **Format width specifier**
>
> `%99s` limits the read to 99 characters (+ null terminator = 100), preventing buffer overflow when the word array is `char word[100]`.

b3.c — Store Student Records to File

Functions: fopen · scanf · fprintf · fclose

Purpose: Accepts multiple student records (Name, ID, Grade) from the user and stores them in `students.txt` using `fprintf()`.

```
struct Student { char name[50]; int id; float grade; };

fp = fopen("students.txt", "w");

for (i = 0; i < n; i++) {
    scanf("%s %d %f", s.name, &s.id, &s.grade);
    fprintf(fp, "%s %d %.2f\n", s.name, s.id, s.grade);
}
```

This program demonstrates writing **structured (record-based) data** in text format. Each record is written as one line with space-separated fields, making it easy to read back with `fscanf()` or parse in other languages.

> **%.2f format specifier**
>
> `%.2f` formats the float to exactly two decimal places, which is important for consistent file formatting when reading records back.

9. Function Comparison at a Glance

Function	Direction	Granularity	Format	Loop condition	Used in
`fgetc()`	READ	1 character	None	`!= EOF`	a2.c, b1.c
`fgets()`	READ	1 line	None	`!= NULL`	a2.c*, a4.c
`fscanf()`	READ	1 token/field	Format string	`== 1` (or field count)	a2.c*, b2.c
`fputc()`	WRITE	1 character	None	—	b1.c
`fputs()`	WRITE	1 string	None	—	—
`fprintf()`	WRITE	1 formatted string	Format string	—	a1.c, a3.c, b3.c
`fopen()`	OPEN	File	Mode string	Check != NULL	All programs
`fclose()`	CLOSE	File	—	—	All programs

* a2.c shows fgets and fscanf in commented-out alternative code blocks.

10. Common Errors & Best Practices

10.1 Always Validate fopen()

```
// ❌ Wrong – no NULL check
fp = fopen("data.txt", "r");
fgetc(fp);   // undefined behaviour if file missing

// ✅ Correct
fp = fopen("data.txt", "r");
if (fp == NULL) { perror("fopen"); return 1; }
```

10.2 Replace gets() with fgets()

```
// ❌ Dangerous (buffer overflow)
gets(str);

// ✅ Safe
fgets(str, sizeof(str), stdin);
```

10.3 Store fgetc() Result in int

```c
// ❌ Wrong — may never equal EOF on unsigned-char platforms
char ch;
while ((ch = fgetc(fp)) ≠ EOF)  ...

// ✅ Correct
int ch;
while ((ch = fgetc(fp)) ≠ EOF)  ...
```

10.4 Use Width Specifiers with fscanf

```c
// ❌ Risky — no width limit
fscanf(fp, "%s", word);

// ✅ Safe — limits input to buffer size - 1
fscanf(fp, "%99s", word);   // for char word[100]
```

10.5 Always Close Files

```c
// After all operations are done:
fclose(fp);   // flushes internal buffer → data guaranteed on disk
```

11. Summary

The seven lab programs collectively cover the complete spectrum of C file I/O operations. The table below maps each program to its primary educational goal:

Program	Core Concept	Primary Functions
a1.c	Writing formatted text to a new file	fopen("w"), fprintf, fclose
a2.c	Three methods to read a file: char / line / word	fgetc, fgets, fscanf
a3.c	Appending to a file without overwriting	fopen("a"), fprintf, fclose
a4.c	Line-by-line search using string functions	fgets, strstr, fclose
b1.c	Character-level I/O; merging multiple files	fgetc, fputc, fclose ×3
b2.c	Token-based reading; word counting	fscanf, fclose
b3.c	Structured record storage using structs	fprintf (multi-field), fclose

Key Takeaways

- Always include `<stdio.h>` for file I/O functions.
- Every file must be opened with `fopen()` and the returned pointer checked for NULL.
- Use `"r"` to read, `"w"` to write (truncates), `"a"` to append.
- Use `fgetc/fputc` for character-level, `fgets` for line-level, `fscanf/fprintf` for formatted I/O.
- Always close files with `fclose()` to flush buffers and free resources.
- Prefer `fgets()` over `gets()` to avoid buffer overflows.

Darshan University | Sem-4 TA (BSC-H) | LAB-16 | C File Handling — Reading & Writing Functions | March 2026